5-Minute Loom Video Script & Blueprint

AI-Assisted Property Address Standardization System — Code Walkthrough

---

Pre-Recording Checklist:
• **VS Code Font Size:** Set to 130%–150% (Ctrl + +) for maximum readability.
• **Project Root:** Open C:\Users\sanat\Desktop\Adress_project2\Adress project.
• **Initial View:** Collapse all folders in the file explorer before recording starts.
• **Audio & Mic:** Speak at a natural, steady pace with quiet background surroundings.

1. Video Timeline & Screen Actions Matrix

Time	Section	Files / Screen Focus	Key Objective
0:00–0:30	1. Introduction	File Explorer (Root Directory)	Hook viewer with problem & system goal
0:30–1:10	2. Project Structure	docker-compose.yml & top-level folders	Explain microservice separation of concerns
1:10–2:10	3. Backend Architecture	addresses.controller.ts, addresses.service.ts	Walk through NestJS flow, queues & audit trail
2:10–3:40	4. ML Service & Pipeline	main.py, parser.py, features.py, model.py	Show libpostal, feature engineering & LightGBM
3:40–4:20	5. Decision Flow	README.md matrix / threshold logic	Confidence routing & active learning loop
4:20–5:00	6. Closing	Entire VS Code Window	Summarize architecture & thank interviewer

2. Section-by-Section Script & Action Blueprint

Section 1: Introduction (0:00–0:30)

■ **Visual: VS Code with root project folder open. Show clean workspace.**

■■ ***Script:*** *"Hi, I'm Sanath. This project is an AI-Assisted Property Address Standardization System. In real-world property databases, raw address inputs are messy, inconsistently formatted, and full of abbreviations. The goal of this system is to ingest raw property addresses, standardize them into a canonical format, predict a machine learning confidence score, and automatically decide whether to accept the result or send it for human review."*

Section 2: Project Structure (0:30–1:10)

■ **Visual: Collapse all folders in VS Code. Point to backend/, ml-service/, database/, docker-compose.yml.**

■■ ***Script:*** *"I separated the system into independent services to keep responsibilities clean and modular:*
• backend/: Built with NestJS. Exposes REST APIs, manages PostgreSQL records, coordinates queues, and maintains audit trails.
• ml-service/: Built with Python FastAPI. Handles address parsing using libpostal, feature extraction, LightGBM scoring, and vector embeddings.
• database/: Contains PostgreSQL migrations and schema with pgvector enabled.
• docker-compose.yml: Orchestrates the entire containerized stack including Redis and PostgreSQL."

Section 3: Backend Architecture (1:10–2:10)

■ **Visual: Expand backend/src/addresses -> Open addresses.controller.ts (lines 13-36) and addresses.service.ts (lines 39-53). Highlight review/ and audit/ folders.**

■■ ***Script:*** *"In backend/src/addresses/addresses.controller.ts, the POST /standardize endpoint accepts raw input. In addresses.service.ts, the service sends raw text to the ML service and receives standardized components plus a confidence score. High confidence matches are saved directly to canonical records, while lower confidence items route into review/. Crucially, every operation logs an immutable record in audit/, giving interviewers complete visibility and auditability."*

Section 4: ML Service & Decision Logic (2:10–3:40)

■ **Visual: Open ml-service/app/main.py -> parser.py -> features.py -> model.py.**

■■ ***Script:*** *"In ml-service/app/main.py, POST /standardize drives the ML pipeline:*
1. parser.py: Uses libpostal to parse raw text into house_number, road, city, state, zip.
2. features.py: Computes Jaccard similarity, Levenshtein edit ratios, directional expansions (W to West), and suffix expansions (St to Street).
3. model.py: A LightGBM Classifier evaluates these features to predict a confidence score between 0 and 1. Predicting confidence—rather than a binary output—gives us flexible control over automation thresholds."

Section 5: Decision Logic & System Flow (3:40–4:20)

■ **Visual: Show README.md routing matrix table or addresses.service.ts lines 44-52.**

■■ ***Script:*** *"The decision architecture routes records dynamically:*
• Score >= 0.80 -> Auto Accepted: Bypasses review and updates canonical records.
• 0.50 - 0.79 -> Review Queue: Routed to human reviewers, prioritized near the decision boundary using uncertainty sampling.
• Score < 0.50 -> Flagged: Immediately escalated for manual intervention.
When reviewers submit corrections, the system collects feedback to trigger automated model retraining."

Section 6: Closing (4:20–5:00)

■ **Visual: Switch back to top-level VS Code view.**

■■ ***Script:*** *"To summarize, this project highlights clean service boundaries, asynchronous ML integration, and a pragmatic human-in-the-loop design. Using confidence scores with an active learning loop ensures we keep bad data out of production while continuously improving the model. Thank you for watching!"*